

BSc and MSc Student Project Portfolio

Industrial projects, graph applications, time-series learning, vision and signal processing

Choose from eighteen project entries across five categories. Each description introduces the goal, data and starting material, approach and degree-level scope. The DINO project offers two application choices; choose one.

Bachelor projects study established methods through a fair comparison. Master projects investigate an extension, such as robustness, transfer to new conditions or a new representation. A well-supported negative result is a valid outcome.

The Siemens Gamesa projects retain their partner-agreed descriptions. Public datasets and reference code are linked within the other entries. Check access conditions and agree a manageable scope with the supervisor before starting.

Industrial projects

These projects are developed with Siemens Gamesa and Tetra Pak using non-public operational data. At the start of each project, the student, supervisor and partner will confirm the available data, downstream task and permitted outputs.

All three are primarily Master projects. A reduced Bachelor version can be offered when a prepared dataset and fixed task are available. The Bachelor project implements and compares established methods; the Master project adds a new representation, feature design or evaluation question.

Evaluation split. Keep all measurements from one turbine or filtration cycle in the same split. A held-out site or production line can be used as an additional Master-level transfer test when enough data are available.

Starting resources

- Wind-turbine vibration analysis: Randall and Antoni, [Rolling element bearing diagnostics — a tutorial](#), *Mechanical Systems and Signal Processing*, 2011.
- Time-series representation learning: [TS2Vec](#), [TS-TCC](#) and [SimMTM](#).
- Time-series baselines: [ROCKET](#) and [MiniRocket](#) and [InceptionTime](#).
- For every project, state how the result would be used: for example, choosing an operating-state model, selecting a drivetrain configuration, or flagging a filtration cycle for review.

1.1 Siemens Gamesa: Data-driven operational state identification for wind turbine condition monitoring

Task: Unsupervised operating-state discovery followed by condition-monitoring evaluation

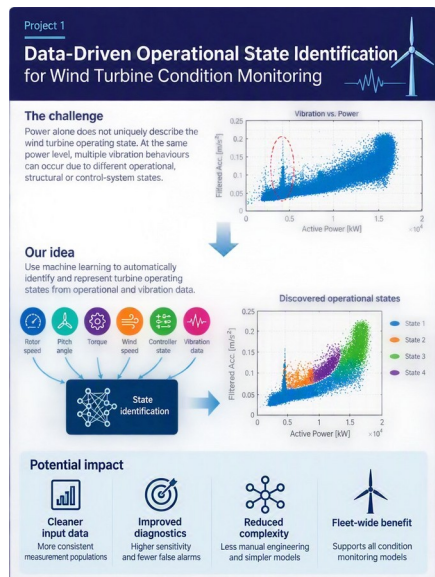
Level: MSc preferred; reduced BSc version possible

Compute: Light to moderate; compact models on a 4–8 GB GPU

Data: Partner-supplied Siemens Gamesa vibration and operational data; not public

Starting paper: Yue et al., [TS2Vec](#), AAAI 2022

What you will do. Wind-turbine monitoring systems often group vibration measurements by power output. However, turbines producing the same power can show different vibration behaviour. You will test whether operating states learned from vibration and operational measurements describe the turbine more consistently than those learned from power bins.



Project overview: operational-state identification from turbine operational and vibration data

Project goal. Discover recurring operating states and determine whether using those states improves a downstream condition-monitoring model. The project should answer whether the learned states contain useful information beyond turbine power. Where possible, the discovered operating states should be interpreted in relation to known turbine operating regimes, controller actions, or structural and aerodynamic phenomena.

Data. Siemens Gamesa will provide anonymised turbine data under an agreed access arrangement. The data may include vibration features or spectra together with variables such as power, rotor or generator speed, pitch angle, torque, wind conditions and controller information. The exact channels, number of turbines, time period and sampling format must be confirmed before allocation.

Methods and code. Start with power binning, PCA and one clustering method such as k-means or a Gaussian mixture. Then test one representation-learning approach, such as [TS2Vec](#), followed by clustering. Use UMAP only for visualisation, not as evidence that the states are valid.

Baselines and evaluation. Compare power bins, clustering on standardised operational variables, and clustering on the learned representation. Evaluate whether states are stable across held-out turbines or time periods and whether they have a clear operational interpretation. The main downstream comparison uses the same condition-monitoring or anomaly model with power-bin conditioning and learned-state conditioning. Report residual variation and, if suitable event or alarm labels are available, detection performance and false-alarm rate.

Master research question. **Do data-driven operating states describe wind-turbine vibration more consistently than power bins, and do they improve downstream condition monitoring?** The contribution is the state representation and a controlled comparison showing when it improves—or does not improve—the downstream model. Include stability analysis and one ablation of the input variables or representation method.

Bachelor version. Using a prepared feature table, compare power binning with one clustering method. Describe the resulting states using the available operational variables and measure how much within-state vibration variation each grouping removes. Representation learning is not required.

Practical notes. Unsupervised clusters are not automatically physical operating states. Do not judge the result only by silhouette score or a two-dimensional plot. A useful state should be repeatable, physically interpretable where possible, and should improve an agreed downstream measure.

1.2 Siemens Gamesa: Probabilistic drivetrain configuration and kinematics identification from vibration signals

Task: Probabilistic ranking of candidate drivetrain configurations from vibration and operational measurements

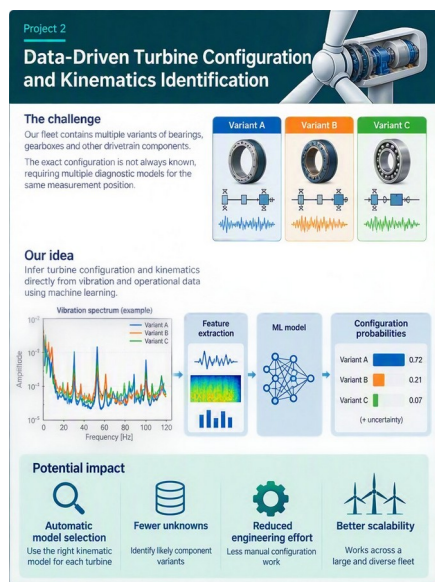
Level: MSc preferred; reduced BSc version possible

Compute: Light to moderate; compact models on a 4–8 GB GPU

Data: Partner-supplied Siemens Gamesa data with configuration information for a subset of turbines; not public

Starting paper: Dempster, Petitjean and Webb, [ROCKET](#), DMKD 2020

What you will do. A wind-turbine fleet can contain several bearing, gearbox or drivetrain variants, and the monitoring system may not always know which configuration is installed. Because component geometry affects the vibration spectrum, you will test whether measured vibration and operational data can narrow the set of plausible configurations and provide probabilities or confidence estimates for the candidates.



Project overview: turbine-configuration and kinematics identification from vibration data

Project goal. Use vibration and operational measurements to determine whether fixed drivetrain hardware variants can be distinguished and, when candidate configurations are known, rank the most plausible configurations for each turbine. Verified configuration information for a subset of turbines or components will be used to validate and calibrate the rankings. As an optional physics-informed extension, condition the model on known bearing or drivetrain kinematics and test whether relationships learned from well-documented components can be transferred to more complex components with limited labels.

Data. Siemens Gamesa will provide anonymised vibration and operational measurements, potentially with verified configuration information for a subset of turbines. Before allocation, confirm the number of turbines and configuration classes, how the configuration labels were obtained, the measurement positions, sampling

format, and whether raw waveforms, spectra or extracted features are available. The probabilistic ranking study requires enough independent turbines for each candidate configuration.

Method and code. Extract spectral features from the vibration measurements and combine them with available operational data. Train a simple probabilistic model, such as [multinomial logistic regression](#), to assign a probability to each candidate drivetrain configuration. If raw spectra or time series are available, compare it with [MiniRocket](#), for which [open-source code](#) is available. As a physics-informed extension, include available bearing or drivetrain kinematic information in the model and test whether it improves the configuration ranking, particularly for components with limited labelled data.

Evaluation: Compare models using operational data only, vibration features only, and both together, and evaluate them on turbines not used for training. Assess whether the correct configuration is ranked first or among the leading candidates and whether the reported confidence is reliable.

Master research question. Can vibration and operational measurements narrow the set of plausible drivetrain configurations and provide calibrated confidence estimates beyond what operational metadata already reveals? The contribution may be a calibrated candidate-ranking method, a physics-informed feature set or a model conditioned on kinematic information. Include a metadata-only control and analyse performance across operating conditions.

Bachelor version. Using a prepared feature table and verified configuration labels, compare one probabilistic classifier with the metadata-only baseline on turbine-grouped splits. Report both the highest-ranked classification result and whether the true configuration appears among the top candidate configurations. Identify the most informative spectral or operational features.

1.3 Tetra Pak: Learning from filtration cycles

Level: MSc preferred; reduced BSc version possible

Computing: A compact model is sufficient; requirements depend on the supplied data.

Goal

Learn useful patterns from filtration-process measurements, such as pressure, flow and energy use. Investigate whether accounting for the stage of a production or cleaning cycle helps distinguish normal operating changes from inefficient or abnormal behaviour. Choose one prediction or detection task with the industrial partner.

Data and starting material

Tetra Pak will provide process data under an agreed access arrangement. The available measurements, complete cycle count and target to predict must be confirmed before allocation.

- [TS2Vec paper and authors' code](#).

Methods and evaluation

Explore simple statistical models, time-series neural networks and self-supervised learning, which learns from recordings without requiring labels for every example. Compare an established approach with one that accounts for cycle stages or operating conditions.

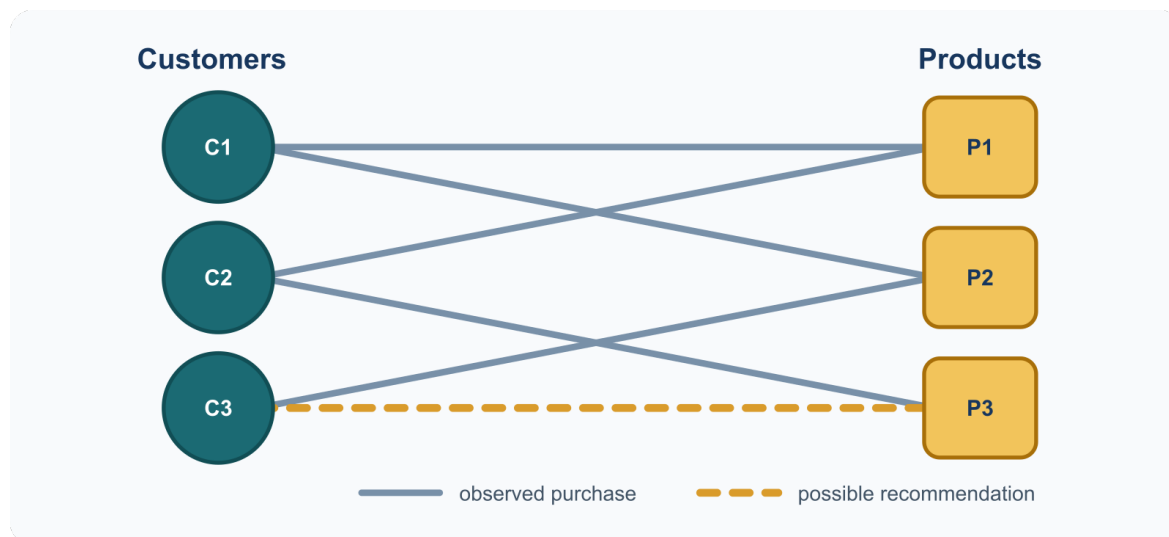
Evaluate on complete later cycles kept separate from training. Measure prediction error or detection quality for the agreed task, and check whether using cycle information improves the result.

Bachelor focus. Using prepared cycles and a fixed task, compare one simple reference method with one established self-supervised method.

Master extension. Develop a way to account for cycle stage or operating conditions during learning. Test whether it improves performance on different regimes or production lines when suitable data are available.

GNN applications

A **graph** is a way to represent entities and the relationships between them. The entities are called **nodes** and their relationships are called **edges**. Both may carry useful information: a customer node might contain age or location, a product node might contain category and price, and an edge might record that a customer bought or rated a product. Road sensors connected by roads, transactions linked by payments and atoms connected by chemical bonds are other examples.



A small customer-product graph. Solid edges are observed purchases; the dashed edge is a possible recommendation.

A **graph neural network (GNN)** learns numerical representations, or embeddings, by combining a node's own features with information from its neighbours. In the example, the model can use a customer's previous purchases, the properties of products and patterns from nearby customers to score a new customer-product link. The same principle supports **node classification** (classify a transaction), **link prediction** (recommend a product), and **graph-level prediction** (predict a property of an entire molecule or sensor system).

Each project asks when relationships between entities help prediction. Students begin with one simple reference model and one graph model, then investigate the value of the graph.

Starting resources

- [A Gentle Introduction to Graph Neural Networks.](#)
- [Stanford CS224W course and project guidance.](#)
- [PyTorch Geometric code and examples.](#)

2.1 Predicting the remaining life of aircraft engines

Level: BSc or MSc

Computing: Start with a small model; a GPU can help with training.

Goal

Predict how many operating cycles remain before a simulated aircraft engine fails. Investigate whether sharing information between related sensor channels improves predictions, and how the choice of sensor connections affects the result.

Data and starting material

Use the FD001 subset of [NASA C-MAPSS](#), which contains simulated engine-degradation measurements from 21 sensor channels. It provides 100 training engines and 100 test engines.

- [Spatio-Temporal Attention GNN for Remaining Useful Life Prediction](#).
- [GNN remaining-life benchmark code](#).

Methods and evaluation

Explore time-series neural networks and spatiotemporal graph neural networks, which combine changes over time with relationships between sensors. Start with one model using sensor histories and one graph model.

Keep complete engines separate between training and evaluation, and measure the error in predicted remaining life. Compare meaningful sensor connections with shuffled connections and a version without information sharing.

Bachelor focus. How much does the choice of connections between sensors affect remaining-life prediction?

Master extension. Explore a model that adjusts sensor connections as an engine ages. Test whether it handles noisy or missing sensor measurements better than fixed connections.

2.2 Predicting traffic speed with graph neural networks

Level: BSc or MSc

Computing: A GPU is recommended; requirements depend on the chosen model.

Goal

Predict traffic speed at road sensors **15, 30 and 60 minutes ahead**. Investigate whether using information from connected sensors improves predictions compared with using each sensor's own history, and whether this becomes more useful when forecasting further ahead. Each sensor represents a location in a graph; its connections allow the model to learn how traffic conditions at different locations relate to one another.

Data and starting material

Use **METR-LA**, which contains traffic-speed measurements from 207 sensors in Los Angeles, recorded every five minutes.

- [Dataset access and preparation instructions](#).
- [Graph WaveNet paper and authors' code](#).

Methods and evaluation

Explore simple forecasting methods, time-series neural networks and **spatiotemporal graph neural networks**, which learn from both changes over time and relationships between sensors. Start with a simple reference method and one graph model.

Evaluate predictions on later data that the models have not seen, using average prediction error at the three forecasting horizons. Compare meaningful sensor connections with shuffled connections and a version without information sharing between sensors to establish when the graph helps.

Bachelor focus. Does information from connected sensors become more useful when predicting 60 minutes ahead than when predicting 15 minutes ahead?

Master extension. Explore a model that adjusts sensor connections as traffic conditions change. Test whether this improves 60-minute forecasts compared with connections that stay fixed, including during rush hour.

2.3 Detecting illicit Bitcoin transactions with graphs

Level: BSc or MSc

Computing: A GPU is useful for graph models; begin with a small experiment.

Goal

Identify transactions labelled as associated with illicit activity. Investigate whether payment connections improve classification, and whether the model remains useful when some connections are missing.

Data and starting material

Use the [Elliptic Bitcoin dataset through PyTorch Geometric](#). It provides transaction features, payment connections and labels for a subset of transactions. Transactions with unknown labels must remain unlabelled.

- [Dataset paper: Anti-Money Laundering in Bitcoin](#).
- [PyTorch Geometric code and examples](#).

Methods and evaluation

Explore conventional classifiers, such as random forests, and graph neural networks that use neighbouring transactions. Include a comparison using only the transaction's own features, since some supplied features already summarise neighbours.

Train on earlier transactions and evaluate on later ones. Measure how well the models find illicit transactions while limiting false alarms, and repeat the comparison after removing some payment connections.

Bachelor focus. How much does missing payment information affect graph-based detection?

Master extension. Explore a model that accounts for changes over time or estimates uncertainty. Test whether its predictions remain reliable as transaction patterns change.

2.4 Recommending movies with graph neural networks

Level: BSc only

Computing: A CPU or a small GPU is suitable for initial experiments.

Goal

Recommend films that a user has not yet rated. Investigate whether a graph of users and their ratings improves recommendations, and whether better accuracy comes at the cost of recommending a narrower range of films.

Data and starting material

Use [MovieLens 100K](#), which contains 100,000 ratings from 943 users on 1,682 movies. The project uses ratings to identify films users like.

- [LightGCN paper and authors' code](#).

Methods and evaluation

Explore popularity-based recommendation, matrix factorisation and graph-based recommendation, such as LightGCN. These approaches respectively use overall popularity, shared user–film patterns and connections in the rating graph.

Evaluate on later ratings kept out of the training graph. Compare recommendation quality and how widely the recommendations cover the film catalogue, including less popular films.

Bachelor focus. Does graph-based recommendation improve accuracy while still exposing users to a broad range of films? Study one factor, such as how far information travels through the graph.

2.5 Predicting molecular activity with graph neural networks

Level: BSc or MSc

Computing: A GPU is recommended for the graph models.

Goal

Predict whether a molecule is active in an HIV screening assay. Investigate whether representing atoms and bonds as a graph helps the model generalise to molecular structures different from those used for training.

Data and starting material

Use [OGB ogbg-molhiv](#), which contains 41,127 molecular graphs and an official split based on molecular structure. This is an assay-prediction task.

- [Open Graph Benchmark paper and code](#).
- [RDKit chemical-feature tools](#).

Methods and evaluation

Explore conventional classifiers using molecular fingerprints, which summarise chemical structure, and graph neural networks using atoms and bonds. Start with one fingerprint model and one graph model.

Use the official dataset split to evaluate on different molecular structures. Compare how well models distinguish active from inactive molecules and examine where their predictions fail.

Bachelor focus. Do graph neural networks generalise to new molecular structures better than traditional molecular fingerprints?

Master extension. Explore methods that make prediction confidence more reliable on unfamiliar structures. Test whether improved confidence estimates preserve classification performance.

Machine learning for time series

A **time series** is an ordered sequence of measurements. A wearable device may record acceleration every fraction of a second, an ECG contains voltage measurements from several leads, and an industrial plant records pressure, flow, temperature and energy over time. Machine-learning models can use these signals for classification, forecasting, anomaly detection and condition monitoring.

Many projects in this category use **self-supervised learning (SSL)** because useful labels may require expert work and are often scarce. SSL creates a training task from the unlabelled recordings: a model might align two valid views of a signal, reconstruct a hidden segment or predict a future representation. Other projects train directly for forecasting and study how a model represents time, channels and relationships among variables.

Choose a task that interests you: recognising activities, understanding health or industrial signals, or forecasting future measurements. Each project offers a focused comparison rather than requiring several new models at once.

Starting resources

- [TS2Vec paper and code](#) for learning from unlabelled recordings.
- [SimMTM paper and code](#) for masked time-series learning.
- [Mamba code and installation requirements](#) for sequence modelling.

3.1 Recognising activities from wearable sensor data

Level: BSc only

Computing: Start with a small encoder; a GPU can help with training.

Goal

Recognise activities such as walking, sitting and climbing stairs from smartphone motion measurements. Investigate whether learning from unlabelled recordings reduces the amount of labelled data needed to train an activity recogniser.

Data and starting material

Use [UCI Human Activity Recognition](#), which contains motion recordings from 30 people performing six activities. Keep the supplied separation of people between training and test data.

- [TS-TCC paper and authors' code](#).

Methods and evaluation

Explore self-supervised time-series learning, especially contrastive learning, which encourages different valid views of the same recording to have similar representations. Compare it with a small neural network trained directly from activity labels.

Evaluate recognition on people not used for training. Compare performance with small and larger labelled training sets to determine how much labelled data pretraining can replace.

Bachelor focus. How many activity labels can be saved through self-supervised pretraining? Include a comparison without pretraining and explain which activities are hardest to recognise.

3.2 Learning from ECG recordings with fewer labels or leads

Level: MSc preferred; BSc with a supplied data pipeline

Computing: A GPU is recommended; begin with the lower-resolution recordings.

Goal

Learn useful patterns from 12-lead ECG recordings and use them to classify recorded diagnostic categories. Investigate whether learning to recover hidden leads helps when few labels or fewer leads are available.

Data and starting material

Use [PTB-XL](#), which provides 12-lead ECG recordings and diagnostic annotations. Use its official patient-aware training, validation and test folds. Smaller label budgets will simulate limited annotation availability.

- [SimMTM paper and authors' code](#).
- [PTB-XL preprocessing and supervised benchmark](#).

Methods and evaluation

Explore supervised time-series models and masked self-supervised learning, in which the model learns from hidden parts of a recording. For the Master route, compare hiding complete leads with hiding short time segments.

Evaluate on patients kept separate from training. Compare diagnostic classification with different amounts of labelled data or available leads, and examine performance across diagnostic categories.

Bachelor focus. Using the supplied pipeline, study how classification changes when fewer ECG leads are available. Compare supervised training with one established pretraining method.

Master extension. Develop lead-aware masking and test whether it improves learning from limited labels compared with ordinary temporal masking.

3.3 Transferring learned representations between energy time series

Level: BSc or MSc

Computing: A GPU is useful; start with an hourly dataset and a small model.

Goal

Learn patterns from one electricity-transformer time series and reuse them to forecast another. Investigate whether self-supervised pretraining improves predictions when only a small amount of data is available from the new transformer.

Data and starting material

Use the hourly ETTh1 and ETTh2 subsets of the [Electricity Transformer Temperature dataset](#). They record oil temperature and load measurements from different transformers.

- [CoST paper and authors' code](#).
- [DLinear forecasting reference](#).

Methods and evaluation

Explore self-supervised time-series learning and simple forecasting models. Start with representations that capture trend and seasonal patterns, and compare them with a forecaster trained directly on the target series.

Train on earlier periods and evaluate predictions on later periods. Compare forecast error with and without transfer, using the same forecast horizons and target-data availability.

Bachelor focus. Does pretraining on one transformer improve forecasting for another, especially when target training data are limited?

Master extension. Develop a way to account for changing seasonal patterns during pretraining. Test whether it improves transfer when patterns shift over time.

3.4 Learning heat-pump behaviour from smart-meter data

Level: MSc preferred; reduced BSc version possible

Computing: A compact model is suitable after preparing the household recordings.

Goal

Learn patterns in household electricity use and investigate whether they help identify a heat-pump attribute or an issue recorded during an inspection. Test whether unlabelled households can improve predictions for a smaller group with labels.

Data and starting material

Use [HEAPO on Zenodo](#), which provides electricity recordings from 1,408 Swiss households, weather information and inspection data for a smaller subset. Select one target after checking how many households have usable labels.

- [HEAPO paper and dataloader](#).
- [TS2Vec representation-learning code](#).

Methods and evaluation

Explore self-supervised time-series learning and conventional classifiers using consumption summaries, weather or metadata. Compare learning from unlabelled recordings with training directly on the available labels.

Keep each household entirely within one data partition. Compare classification on unseen households with different amounts of labelled data, and report uncertainty when the labelled sample is small.

Bachelor focus. Use a well-covered metadata label, such as heat-pump type, to compare supervised and self-supervised learning.

Master extension. Explore learning that accounts for weather or operating cycles. Test whether it improves prediction for the smaller labelled household group.

Practical note. The linked HEAPO dataset is released under CC BY 4.0. Credit the dataset authors and confirm that the selected target has enough labelled households before allocation.

3.5 Learning from graphs built from bearing vibration signals

Level: MSc only

Computing: Signal preparation is the main initial workload; use a compact graph model.

Goal

Turn bearing vibration recordings into graphs whose nodes describe useful frequency regions. Investigate whether relationships informed by bearing behaviour, together with learning from unlabelled recordings, help recognise faults under different speeds or loads.

Data and starting material

Use the [Paderborn Bearing Data Center](#), which provides measurements from 32 bearings under four operating conditions. Each signal window becomes one graph.

- [GraphMAE paper and authors' code](#).

Methods and evaluation

Explore signal-processing features, graph neural networks and masked graph learning, which learns by recovering hidden graph features. Compare a graph based on bearing knowledge with one based on statistical similarity.

Keep measurements from each physical bearing separate between training and test sets. Compare fault recognition across operating conditions, and test the separate value of graph connections and self-supervised pretraining.

Master extension. Develop a graph representation that accounts for changing bearing speed. Test whether it transfers between operating conditions better than similarity-based graphs, and whether pretraining provides an additional benefit.

3.6 Learning bearing faults with time–frequency models and Mamba

Level: MSc preferred; BSc with prepared data and a fixed comparison

Computing: A GPU is recommended; check the Mamba software requirements early.

Goal

Learn from both vibration waveforms and their frequency content to recognise bearing faults. Investigate whether a Mamba sequence model improves transfer between operating conditions when only a small labelled dataset is available.

Data and starting material

Use the [Paderborn bearing dataset](#). The TF-C repository links to prepared fault-diagnosis data that can support initial experiments; verify their bearing or recording identities before using them for transfer claims.

- [TF-C paper, code and prepared-data instructions](#).
- [Mamba sequence-model code](#).

Methods and evaluation

Explore time–frequency contrastive learning and sequence models such as convolutional networks and Mamba. Begin with the established TF-C approach and change one encoder component.

Compare fault recognition across operating conditions and with different amounts of labelled data. Keep related recordings separate, and measure both recognition quality and computational cost.

Bachelor focus. Using prepared data, does replacing the existing encoder with Mamba improve recognition or efficiency under the same learning setup?

Master extension. Develop a signal view that accounts for bearing frequencies or changing speed. Test whether it improves transfer to different conditions and, where identities are available, unseen bearings.

3.7 Using relationships between signals to improve Mamba forecasts

Level: MSc preferred; reduced BSc evaluation version possible

Computing: Start with Weather; Electricity requires a larger experiment.

Goal

Forecast several related measurements together, such as weather variables or electricity demand. Investigate whether explicitly connecting related signals improves a Mamba forecaster and makes its predictions less sensitive to the order in which variables are presented.

Data and starting material

Start with the Weather benchmark available through [Autoformer](#). The Electricity benchmark is a possible follow-up once the first experiment works.

- [MambaTS paper and authors' code](#).
- [DLinear forecasting reference](#).

Methods and evaluation

Explore simple forecasting methods, Mamba sequence models and graph-based sharing of information between variables. For the Master route, build connections using relationships estimated from training data.

Evaluate forecast error on later unseen data. Compare predictions when the variable order changes or some measurements are missing, using consistent changes to inputs and outputs.

Bachelor focus. How sensitive are existing Mamba forecasters to variable order, and does modelling relationships between variables help?

Master extension. Develop a graph module connecting related signals. Test whether it improves forecasting and robustness compared with models that process channels independently or mix them sequentially.

Deep-learning vision models

A **vision foundation model** learns useful image features from a large image collection. These features can be reused for specialised tasks without training a foundation model from scratch.

Choose between adapting DINO for industrial inspection or medical segmentation, and a separate Master project on federated image denoising. In the DINO project, choose one application. The denoising project uses image-restoration networks such as U-Net.

Starting resources

- [DINOv2 introduction, model card and code.](#)
- [DINOv3 model access and code.](#)
- [MONAI tools for medical-image learning.](#)

4.1 Adapting DINO vision models for specialised image analysis

Level: BSc or MSc

Computing: A GPU is recommended; medical-image preparation and adaptation need more resources.

Goal

Investigate how well a pretrained DINO model works on a specialised image task, and whether adapting it improves accuracy and reliability. **Choose one application** below; students are not expected to complete both.

Data and starting material

- **Industrial inspection:** Detect and locate defects in product images using [MVTec AD](#). Learn from defect-free training images and evaluate on unseen normal and defective images.
- **Medical imaging:** Identify abdominal organs in CT scans using the CT portion of [AMOS22](#). Learn from scans with labelled organ boundaries and keep patients separate between training and evaluation.

Starting resources for the selected application:

- [AnomalyDINO paper and DINOv2 detection code](#).
- [DINOv3 model access and code](#).
- [MedDINOv3 medical-segmentation example](#).

Methods and evaluation

Start by using DINO's pretrained features without changing the backbone. For the Master route, explore a suitable adaptation method, such as training small additional modules or fine-tuning selected layers. Choose the task-specific detector or segmentation decoder for the selected application.

Compare performance on unseen images with a conventional reference method and, for the Master route, before and after adaptation. Investigate how changes in lighting or image quality affect detection or segmentation, using measures appropriate to the chosen task.

Bachelor focus. Study the pretrained features on one application and identify where they work well or fail. Train only the task-specific decoder when segmentation requires it.

Master extension. Investigate whether adaptation improves robustness on the chosen application, and assess the improvement relative to its computational cost.

Practical note. Begin with the backbone used by the selected reference implementation. Confirm dataset access and model-weight access before allocation; DINOv3 weights require access approval and acceptance of the model terms.

4.3 Federated learning for medical-image denoising

Level: MSc only

Computing: A GPU is recommended; simulated clients can be trained sequentially.

Goal

Simulate hospitals that improve MRI image quality while keeping their local scans separate. Investigate whether sharing model predictions on a common image set can train a useful global denoising model without sending local scans or model weights to a central server.

Data and starting material

Use [M&Ms cardiac MRI](#) to form simulated clients based on scanner vendor, and [ACDC](#) as a shared reference-image set. Add controlled synthetic noise and use the original images as reconstruction targets.

- [Federated distillation reference code \(FedDF\)](#).
- [Flower federated-learning framework](#).

Methods and evaluation

Explore image-restoration networks such as U-Net, federated learning and knowledge distillation, which trains a student model from other models' predictions. Adapt prediction sharing to image denoising and compare it with local training and federated weight sharing.

Evaluate image quality on patients kept separate from training, including different scanner vendors or noise conditions. Compare denoising performance and communication cost; conclusions concern removal of the added noise.

Master extension. Can prediction sharing produce a denoising model that works across scanner domains? Study one factor, such as the shared-image set size or differences between client models.

Practical note. Confirm dataset access conditions before allocation. Prediction sharing alone does not provide a formal privacy guarantee.

Signal processing

Many signal-processing methods repeat a fixed optimisation step until convergence. **Algorithm unrolling**, also called **deep unfolding**, turns a fixed number of those iterations into neural-network layers. The update equations determine the network structure, while selected algorithm parameters are learned from data.

This Master-level topic combines signal processing, optimisation and machine learning. Students first establish a correct classical method, then investigate whether learned updates improve its performance.

Starting resources

- [Algorithm Unrolling](#), a survey of the approach.
- [Multivariate Variational Mode Decomposition](#), the classical multichannel method.

5.1 Learning to separate noisy multichannel signals

Level: MSc only

Computing: Begin with small synthetic signals; a GPU is useful for the learned model.

Goal

Separate several related signals into simpler oscillating components. Turn multivariate variational mode decomposition (MVMD), a classical iterative method, into a trainable network and investigate whether it can work faster while preserving decomposition quality when some channels are noisy.

Data and starting material

Begin with synthetic multichannel signals whose components are known. Use the San Diego subset of [LargeST](#), containing 716 traffic sensors, as a later test of whether the decomposition helps forecasting.

- [Classical MVMD Python implementation](#).
- [Adaptive deep-unfolded VMD paper](#).

Methods and evaluation

Explore classical signal decomposition and deep unfolding, which turns a fixed number of algorithm iterations into trainable layers. Extend a single-channel approach to linked channels, and investigate weighting channels according to the reliability of their measurements.

Compare the recovered components with the known synthetic signals, measuring decomposition error, robustness and runtime. Then test whether the method improves traffic forecasting using the same forecaster for each comparison.

Master extension. Can a trainable multichannel decomposition match the classical method, and does reducing the influence of corrupted channels improve recovery?

Practical note. This project requires a strong background in signal processing and optimisation. Establish correct classical decomposition on synthetic data before adding learned updates or the traffic dataset.

Selection guidance

Choose a level

Eight standard projects support Bachelor work: 2.1–2.5, 3.1, 3.3 and 4.1. Projects 2.4 and 3.1 are Bachelor-only; the others also offer a Master extension. In Project 4.1, choose either industrial inspection or medical segmentation.

Projects 3.2, 3.4, 3.6 and 3.7 prefer Master students, with reduced Bachelor versions when a prepared data pipeline or fixed comparison is available. **Projects 3.5, 4.3 and 5.1 are Master-only.**

The three industry projects in Category 1 prefer Master students. A reduced Bachelor version depends on the partner supplying prepared data and an agreed task.

Choose a topic

1. Start with an application you find interesting, such as traffic, health, industrial monitoring or recommendation.
2. Read the goal and choose one focused question you want to answer.
3. Check access to the data, model weights and starting code.
4. Run a simple reference method before planning an extension.
5. Agree the scope and computing needs with the supervisor.

Bachelor work can make a useful contribution through a careful comparison and a clear explanation of the results. Master work should investigate an extension, such as improved robustness, transfer to new conditions or a new representation. An unchanged run of a published method is a starting point for either route.

Data and model access

The project links point to the dataset providers and reference implementations. Public access to a page or repository does not guarantee permission to redistribute its data or model weights. Check the terms on the linked source before using or sharing a resource.

- **Industry data:** Siemens Gamesa and Tetra Pak data are supplied under an agreed access arrangement.
- **Medical data:** Check registration and use conditions for AMOS22, M&Ms and ACDC. PTB-XL provides its access conditions on PhysioNet.
- **DINO models:** DINOv2 has publicly available weights. DINOv3 model weights require an access request and acceptance of the model terms.
- **HEAPO:** The linked Zenodo dataset is released under CC BY 4.0; credit its authors. Check the number of usable household labels before choosing the task.
- **Starting repositories:** Check the supported software versions and any licence before adapting code. A repository link is a reference, not a guarantee that the code will run unchanged in a current environment.

Plan a fair evaluation

Use the same data and evaluation setup for the methods you compare. Keep all related recordings from one person, patient, household, engine, bearing or process cycle together. For forecasting, use earlier data for training and later data for evaluation.

Use validation data to choose model settings and keep the final test set separate. Measure the outcome that matches your goal, and explain where the method works well or fails. A result showing no improvement is valuable when the comparison is fair and the explanation is supported by evidence.

Start with one application, one dataset and a small set of methods. Add further experiments only after the first comparison works.